CDS524 Assignment 1

Reinforcement Learning Game Design Report

Student Name: WANG Shiyu

Game Title: AI Snake — Deep Q-Learning with Dreamy Pink Theme

Submission Date: 25 February 2026

1. Introduction

This project presents an enhanced Snake game developed with Pygame and trained using Deep Q-Learning (DQN). The objective is to control a snake that grows by consuming food while avoiding collisions with the walls and its own body. The dynamic nature of the snake's body creates a challenging decision-making environment that is ideal for reinforcement learning.

Deep Q-Learning (DQN) was employed to enable the agent to learn optimal policies through interaction with the environment. The implementation uses PyTorch for the neural network and experience replay mechanism, while Pygame provides a visually appealing and interactive user interface featuring a dreamy pink theme.

2. Game Design

2.1 Objective and Rules

- The game operates on a 20 × 20 grid with a 900 × 900 pixel window.
- The snake begins at length 3 in the center, facing right.
- At each timestep, the agent chooses one of three relative actions: turn left, go straight, or turn right.
- Collision with walls or the snake's own body results in game over.
- The primary goal is to maximize the score by eating as much food as possible while surviving as long as possible.

2.2 State Space

The environment is represented by an 11-dimensional binary feature vector. It captures (i) immediate collision risk, (ii) food direction relative to the head, and (iii) the current movement direction.

Table 1 provides a precise definition of each dimension (all features are binary: 1=true, 0=false).

Dim	Name	Meaning	Values
0	Danger Straight	Obstacle immediately in front of the head (wall or body)	1=danger, 0=safe
1	Danger Right	Obstacle immediately to the right of the head	1=danger, 0=safe
2	Danger Left	Obstacle immediately to the left of the head	1=danger, 0=safe
3	Food Left	Food is to the left of the head	1=yes, 0=no

4	Food Right	Food is to the right of the head	1=yes, 0=no
5	Food Up	Food is above the head	1=yes, 0=no
6	Food Down	Food is below the head	1=yes, 0=no
7	Move Left	Current movement direction is left	1=yes, 0=no
8	Move Right	Current movement direction is right	1=yes, 0=no
9	Move Up	Current movement direction is up	1=yes, 0=no
10	Move Down	Current movement direction is down	1=yes, 0=no

Table 1: Definition of the 11D state vector

Note: 'left/right/up/down' are defined in screen coordinates relative to the head position (not relative to the current direction).

This compact representation avoids the full grid as input while preserving key information needed for safe navigation and food seeking.

2.3 Action Space

The action space consists of 3 discrete relative actions:

- 0: Turn left
- 1: Go straight
- 2: Turn right

Relative actions naturally prevent invalid 180-degree turns.

2.4 Reward Function

The reward function is designed to encourage desired behaviors:

- Normal food: +10
- Bonus food: +20 + combo multiplier
- Poison food: -8
- Collision (death): -10
- Survival per step: +0.1
- Moving away from food: -0.5 (light shaping reward)

Reward design was refined iteratively. A sparse reward (food/death only) makes exploration inefficient, so a small living reward (+0.1) encourages movement and discovery of useful trajectories. To reduce unnecessary wandering, a light shaping penalty (−0.5) is applied when the snake moves away from the food. Finally, adding bonus and poison food types enriches the learning signal while keeping the core objective unchanged.

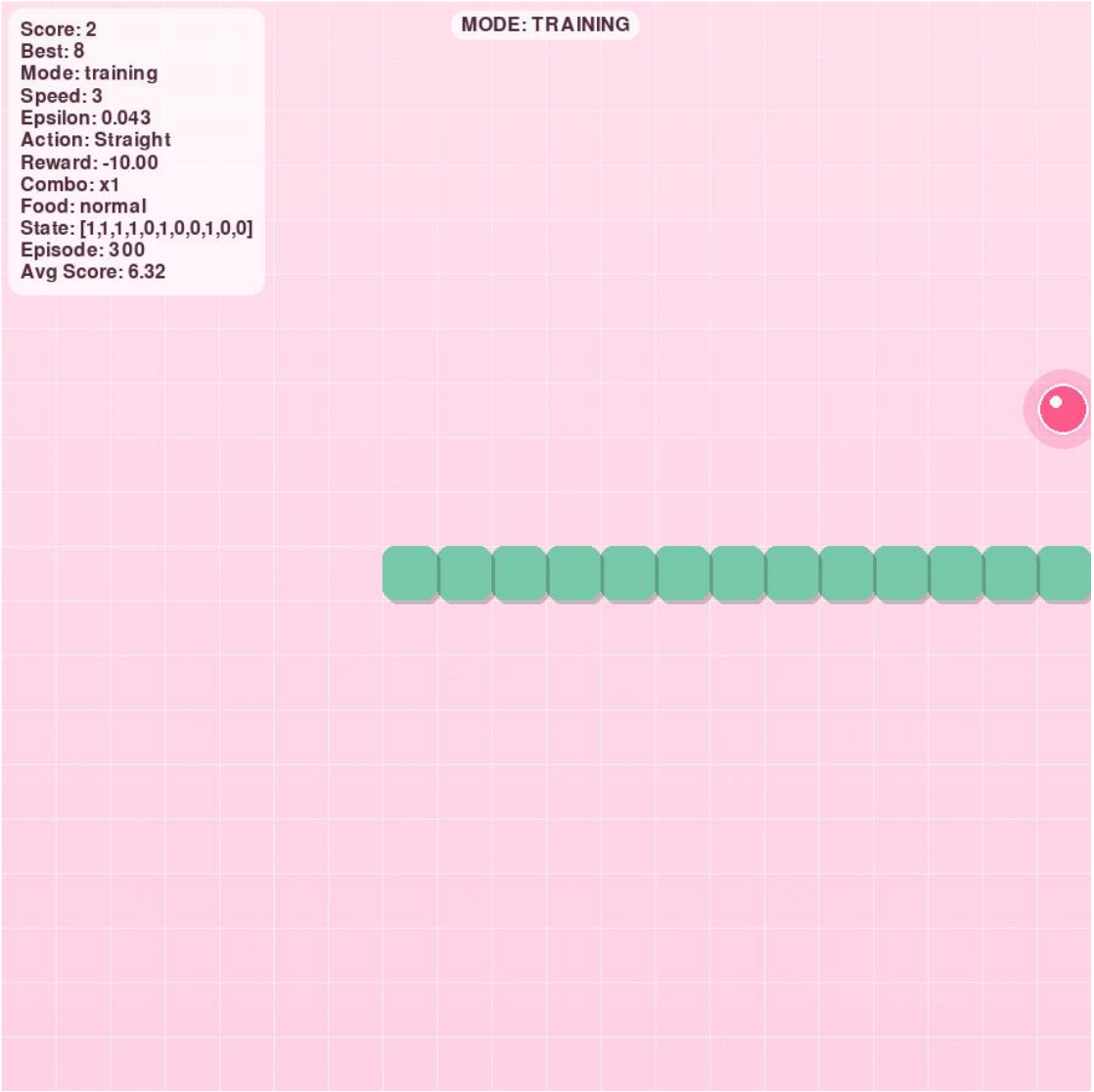

Figure 1: Game interface showing dreamy pink theme, glowing food, combo counter, and real-time 11D state display

3. Q-Learning Implementation

3.1 Algorithm

Deep Q-Learning approximates the Q-value function with a neural network. The target value is computed as:

$$y = r + (1 - \text{done}) \times \gamma \times \max Q(s', a')$$

The network minimizes the mean squared error between predicted and target Q-values.

3.2 Neural Network Architecture

LinearQNet is a 4-layer fully connected network with ReLU activations:

$$11 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$$

The input is the 11D state vector and the output is the Q-value for each of the 3 relative actions (left/straight/right). The hidden layer widths decrease (256→128→64) to progressively compress features into higher-level abstractions while controlling model capacity. This structure is a common, stable baseline for small structured state spaces and converges reliably in this task.

3.3 Exploration vs Exploitation

An ε-greedy policy is used:

- Initial $\varepsilon = 0.8$
- Minimum $\varepsilon = 0.01$
- Multiplicative decay of 0.995 per episode

3.4 Training Components

- Replay buffer capacity: 10,000 transitions
- Batch size: 32
- Optimizer: Adam (learning rate = 0.001)
- Discount factor: $\gamma = 0.95$
- Training episodes in “training” mode: 300

The trained model is saved as model.pth.

4. Game Interaction and User Interface

The game offers a highly polished, intuitive interface with:

- Dreamy pink gradient background and glowing food effects
- Rounded snake blocks with fading trail
- Real-time information card displaying score, best score, mode, ε-value, last action, last reward, combo, food type, and full 11D state vector
- Four interactive modes (manual / random AI / training / test) switched with T (cycle) or F1–F4 (direct jump)
- Sound effects for eating, bonus, poison, death, and pause
- Smooth Game Over screen with “Press Enter to play again”

These features provide excellent user experience and clearly display state, actions, and rewards as required.

5. Evaluation Results

Training progress is evaluated using the episode score trajectory and a moving-average curve (Figure 2). Because the policy is trained under ε-greedy exploration, early episodes are noisy; as ε decays, the agent becomes more consistent.

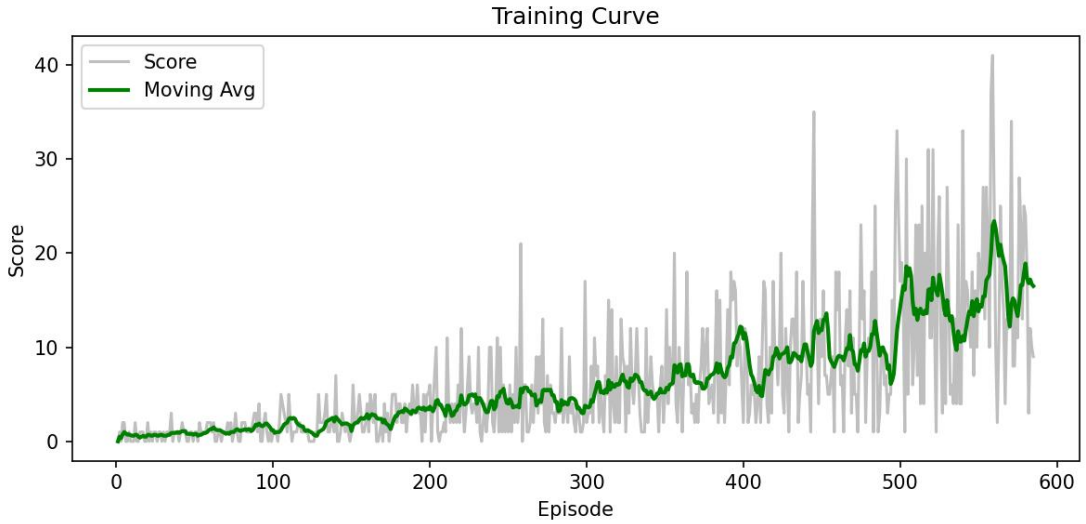

Figure 2: Training curve (training_curve.png) demonstrating learning progress

Key observations from a typical run include:

- Early episodes: low scores with frequent wall/self collisions during exploration.
- Mid training: gradual improvement as the agent learns basic obstacle avoidance and food-seeking behavior.
- Late training: higher moving-average score and fewer immediate collisions as ε approaches its minimum.

In test mode (ε = 0), the trained policy typically survives longer and collects food more reliably than the random baseline, demonstrating that the learned Q-function improves decision quality.

6. Challenges and Solutions

1. Training speed vs visualization

Challenge: Rendering every frame slows training and reduces the number of episodes that can be completed quickly.

Solution: Training runs at high speed, with optional UI refreshes every N episodes; a headless mode disables rendering entirely for notebook/CI environments.

2. Reward shaping trade-offs

Attempt 1: Sparse reward (+food, −death) was simple but made exploration slow because useful trajectories are rare early on.

Attempt 2: A small living reward (+0.1) increased movement and improved exploration stability.

Attempt 3: Stronger directional shaping can speed up learning but may introduce oscillations near the food, so shaping was kept lightweight (penalize moving away only).

Final: Keep the main learning signal simple (+food/−death) and use minimal shaping for efficiency; special food types add variety without changing the core objective.

3. Platform compatibility (Colab/headless)

Challenge: Pygame windows/audio often fail in hosted notebook environments.

Solution: Support HEADLESS=1 with dummy drivers; disable audio and screenshots in headless mode.

4. Presentation quality

Challenge: A plain grid UI is hard to watch in a demo and does not clearly communicate state/action/reward.

Solution: A polished UI (info card + mode badge) shows state/action/reward explicitly; visual theme and sound effects improve demo clarity.

7. Conclusion

This project successfully demonstrates the application of Deep Q-Learning to a dynamic grid-based game. The agent learns effective strategies through trial-and-error, experience replay, and ε-greedy exploration. The combination of a visually stunning interface and measurable learning progress (via the training curve) fully satisfies all assignment requirements.

Future work may include a target network, Double DQN, or pixel-based state representation for even stronger performance.

8. References

- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature.
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.).
- PyTorch Documentation: https://pytorch.org/docs/
- Pygame Documentation: https://www.pygame.org/docs/